GOVERNMENT OF TELANGANA

ONECITIZEN AI — SYSTEM SPECIFICATION & TECHNICAL MANUAL

A Comprehensive Guide to the Citizen Digital Twin Portal, OCR Ingestion Pipeline, Fuzzy Name Verification, and Officer Verification Dashboard

Prepared For:	Gaurav Sujikumar
Contact Email:	gauravsujikumar@gmail.com
Support Hotline:	8498984499
Document Date:	June 2026
System Version:	v2.4.0-Production

1. Executive Summary & Table of Contents

The administrative complexity of citizens interacting with digital governance databases in India often leads to high rejection rates (sometimes exceeding 25% for rural certificate workflows). These rejections are frequently caused by minor mismatches in name spellings across different databases, expired documents, and complex web forms. OneCitizen AI acts as an intelligent intermediary. It offers a secure, unified digital credential vault that acts as a citizen's 'Digital Twin' profile, auto-filling application forms and using AI validation models to check consistency before formal submissions, thus mitigating rejections and streamlining state verification workflows.

Table of Contents

2. System Architecture & Infrastructure

OneCitizen AI is built on a highly reliable, dual-mode microservice infrastructure designed to balance local development speed with robust cloud scalability. It incorporates vanilla frontend components, a centralized Node.js/Express API gateway, and database layers capable of dynamically routing queries based on environmental constraints.

Conceptual Architecture Diagram

Citizen Interface	API Server Gateway	Data & AI Layers
• Vanilla HTML5/CSS3 • ES6+ Asynchronous JS • Leaflet Maps API • Mobile OTP Screen	• Express Route Handler • Multer Ingestion Middleware • JWT Authorization Checks • Twilio SMS Provider	• Gemini 2.0 Flash API • Tesseract OCR Engine • SQLite (Local DB) • Firestore (Cloud DB)

Dual-Mode Storage System:

To support serverless environments (like Vercel) while keeping a simple setup local, the database layer (`backend/db.js` and `backend/firestore.js`) acts as a wrapper. It tests connection states dynamically. In production, if Firestore is available, it commits writes and synchronization steps directly to Google Firestore. If Firestore is blocked or credentials are not found (such as in local dev mode), it catches the error and silently routes operations to the SQLite engine (`one_citizen.db`), ensuring no application crashes occur.

Technical Stack Specifications:

• **Frontend:** Pure vanilla script architecture. Leverages custom CSS variable custom design tokens, keeping load overheads low and maintaining high compatibility. Leaflet.js provides maps for locating physical government service locations.

• **Backend Engine:** Node.js running Express 4.x middleware, coordinating multi-part file uploads and routing prompts to the Gemini API.

3. Citizen Digital Twin Portal Features

The Citizen Portal serves as the primary touchpoint for users. It is designed to act as an offline-first single page application (SPA) where citizens can manage profiles, interact with the Bureaucracy Copilot, locate physical services centers, and submit applications.

Key Functional Modules:

• **Digital Twin Profile:** The center of the application. Once a citizen uploads verification documents, the backend parses these files and saves the metadata. The frontend queries this metadata to automatically pre-fill complex government forms, eliminating typographical and transcription errors.

• **AI Bureaucracy Copilot:** Powered by Gemini 2.0 Flash, this chat assistant acts as a guide. Citizens can talk in natural language (e.g., 'I want to apply for college scholarships but don't know the rules'). The copilot queries the database, matches details against the citizen's profile, determines eligible schemes (like the Telangana Post-Matric Tuition Fee Reimbursement), and inserts direct buttons to launch those application forms.

• **MeeSeva Center Locator:** Integrating Leaflet.js, this map module queries coordinate records from the database and renders physical MeeSeva government offices nearby. Clicking a center provides opening hours, helpline contacts, and geographic directions.

• **Multi-Language Translation System:** To ensure complete accessibility, the entire frontend SPA includes translation tables. With a single dropdown toggle, the interface switches between English, Telugu, Hindi, and Urdu. Elements like button labels, warnings, form placeholders, and toast alerts are hot-swapped dynamically without requiring page refreshes.

• **Secure OTP Login Screen:** Citizen validation is performed through passwordless mobile authentication. Entering a mobile number dispatches a 6-digit OTP which is checked in the backend, preventing unauthorized entry.

4. Document Ingestion & OCR Processing Pipeline

The document vaulting engine verifies all uploaded citizen certificates at ingestion to prevent fraudulent uploads, incorrect formats, or illegible photos. This verification uses a two-tier OCR scanning workflow.

OCR Scanning Sequence:

1. **File Ingestion:** The frontend uploads image files (JPEG, PNG) or PDFs via a multipart form request. Express routes the upload through Multer, writing it to a secure `uploads/` directory on the server disk.

2. **Tier 1 (Gemini Vision OCR):** The server reads the file buffer and sends it as an base64 inline image to the Gemini 2.0 Flash model (`gemini-2.0-flash`). The accompanying system instruction forces the AI to output a strict JSON scheme:

```
"Identify this government document. If it is an Aadhaar card or PAN card, extract the name, date of birth, and document ID number. Output ONLY a valid JSON object matching: { 'document_type': 'aadhaar'|'pan'|'other', 'name': 'Extracted Name', 'dob': 'DD/MM/YYYY', 'id_number': 'XXXX-XXXX-XXXX', 'is_legible': true|false }. Do not write any markdown wrappers or conversational text."
```

3. **Tier 2 (Tesseract.js Fallback):** If the Gemini API fails, is rate-limited, or has no internet access, the backend catches the exception and launches the local `tesseract.js` worker. The engine loads training files (`eng.traineddata` or `tel.traineddata` based on translation state) to perform localized OCR extraction. The raw output string is then processed through regex patterns to extract the fields:

```
Aadhaar regex: [0-9]{4}\s[0-9]{4}\s[0-9]{4} | PAN regex: [A-Z]{5}[0-9]{4}[A-Z]{1}
```

4. **Verification Check:** Once name data is parsed, the server runs a verification check. It compares the extracted name with the user's profile registration name using the Levenshtein distance matching algorithm. If name similarity is below 80%, the file is rejected and deleted, protecting the system against fraudulent uploads.

5. Fuzzy Verification & Readiness Scoring Algorithms

Minor discrepancies (like 'Gaurav Sujikumar' vs 'Gaurav S.') frequently lead to manual rejection in government backoffices. OneCitizen AI implements algorithmic fuzzy name matching and pre-submission checks to resolve this problem.

Fuzzy Name-Matching Mathematics:

The system computes the Levenshtein Distance (d) between two string tokens. The Levenshtein distance is defined as the minimum number of single-character edits (insertions, deletions, or substitutions) required to change one word into another.

Let string A have length m and string B have length n. The Levenshtein distance matrix $D_{i,j}$ is computed as:

$$D_{i,j} = \min \begin{cases} D_{i-1,j} + 1 \\ D_{i,j-1} + 1 \\ D_{i-1,j-1} + \text{cost} \end{cases}$$ where $\text{cost} = 0$ if $A[i] = B[j]$ and $\text{cost} = 1$ otherwise.

The similarity percentage (S) is calculated as:

$$S(A, B) = \left(1 - \frac{d(A, B)}{\max(\text{len}(A), \text{len}(B))}\right) \times 100$$

The system normalizes tokens (converting strings to lowercase, removing punctuation, and stripping titles like 'Sri', 'Kumari', or 'Mr.'). If $S \ge 80\%$, the name is accepted as verified.

Readiness Score Computation:

When applying for a certificate, the backend computes a Readiness Score (R) based on database metadata:

• **Identity Match (40%):** Similarity between profile name and Aadhaar name ($S_{aadhaar}$).

• **Required Uploads (30%):** Fraction of required documents uploaded for the service (F_{docs}).

• **Document Expiry (30%):** Checks if documents (like income or EWS certificates) are valid or expired. Expired files reduce the score to 0.

$$R = 0.40 \cdot S_{aadhaar} + 0.30 \cdot F_{docs} \cdot 100 + 0.30 \cdot \text{Validity}$$

If $R \ge 80\%$, the citizen is allowed to submit the application. Otherwise, submission is blocked, and discrepancies are flagged.

6. Officer Verification Portal Operations

The Officer Portal is a secure dashboard designed for regional verifiers (e.g., Revenue Officers) to manage incoming applications, inspect digital twin profiles, verify attached credentials, and record decisions.

Core Features and Workflows:

• **Work Queue Dashboard:** A clean, grid-based interface. It shows incoming submissions sorted dynamically. Status badges ('Pending', 'Under Review', 'Approved', 'Rejected') use clean, professional outline SVGs to provide clear visibility without cartoony system emojis.

• **Server-Sent Events (SSE) Live Pipeline:** To support high-volume district offices, the dashboard uses an active SSE event pipeline (`/api/admin/applications/live`). When a citizen submits a new application, the backend broadcasts this event. The officer's browser receives it in real-time, updates sidebar badges and queue rows, and displays a clean, browser-native toast notification.

• **Bulk Verification Quick Actions:** Officers can run a bulk scan. The backend checks pending files against secure document vaults, verifies credentials against automated parameters, and highlights applications suitable for immediate bulk approval.

• **Detail Overlay Inspector:** Clicking an application opens a details panel. This panel loads metadata fields, form entries, and documents directly from the database. It displays document verification states ('VERIFIED' in green, 'PENDING' in orange) next to 'View File' buttons. It also displays the newly styled SVG paperclip icon next to 'Certificate Application Uploads'.

• **Decision Workflows:** Officers can record notes and make decisions. Clicking 'Approve' updates the database, moves the certificate to the citizen's vault, and alerts the user. Clicking 'Reject' displays a mandatory reason field, ensuring administrative transparency.

• **Bulk Queue & File Clearing:** Officers can purge the verification queue using a dedicated 'Clear Files' quick action. This permanently clears all applications and documents from the database and deletes uploads.

• **Individual Application Deletion:** When inspecting an application, officers can selectively delete the application using the 'Delete Application' action, which removes related files and document records for that citizen.

7. Backend API Endpoint Reference

The backend Express app exposes REST endpoints designed for integration with client applications, auth services, and AI APIs. All routes require authorization headers unless stated otherwise.

Authentication & OTP endpoints:

• **POST /api/auth/register:** Creates a new citizen user. Automatically hashes passwords and initializes blank profiles. Role defaults strictly to 'citizen'.

• **POST /api/auth/login:** Validates email/password credentials and issues a secure JWT token.

• **POST /api/otp/send:** Dispatches a 6-digit OTP code to the citizen's mobile number. Implements rate-limiting to prevent SMS flood attacks.

• **POST /api/otp/verify:** Checks the submitted OTP code. If correct, issues a JWT token. Invalidation occurs after 5 failed attempts.

Document Vault endpoints:

• **GET /api/documents:** Lists all verified files in the citizen's vault.

• **POST /api/documents/upload:** Ingests a new certificate file. Triggers Multer, routes the buffer to Gemini OCR models, checks name matching, and saves metadata.

• **DELETE /api/documents/:id:** Removes a file from the vault and database.

Service & Application endpoints:

• **GET /api/services:** Lists all government certificates (Income, Caste, EWS, Birth, Death) and their requirements.

• **POST /api/services/apply:** Processes certificate applications. Performs a pre-submission readiness check, compiles metadata, and inserts records into the queue.

Administrative & Queue endpoints:

• **GET /api/admin/applications:** Returns all submissions, sorted by created date.

• **GET /api/admin/applications/:id:** Returns detailed citizen records and documents for the selected application.

• **PATCH /api/admin/applications/:id/status:** Updates status to approved/rejected. Triggers notification alerts.

• **DELETE /api/admin/applications/clear:** Purges all applications, document vault records, and uploaded files on disk.

• **DELETE /api/admin/applications/:id:** Deletes a specific application, its citizen's documents metadata, and deletes physical uploads.

8. Relational Database Schema Design

OneCitizen AI uses a relational schema. SQLite (development) and Firestore (production) fallback wrappers maintain mapping parity. Here is the SQLite schema configuration:

Table: users

Stores authentication credentials and account roles.

```
CREATE TABLE users ( id INTEGER PRIMARY KEY AUTOINCREMENT, email TEXT UNIQUE, password_hash TEXT, role TEXT DEFAULT 'citizen', mobile TEXT UNIQUE, created_at TEXT DEFAULT CURRENT_TIMESTAMP );
```

Table: citizen_profiles

Stores citizen profiles (the Digital Twin) parsed from verified documents.

```
CREATE TABLE citizen_profiles ( user_id INTEGER PRIMARY KEY REFERENCES users(id), name TEXT DEFAULT '', dob TEXT DEFAULT '', gender TEXT DEFAULT '', occupation TEXT DEFAULT '', income_amount REAL DEFAULT 0, state TEXT DEFAULT '', district TEXT DEFAULT '', caste TEXT DEFAULT '', updated_at TEXT DEFAULT CURRENT_TIMESTAMP );
```

Table: documents

Stores metadata of documents verified through the OCR pipeline.

```
CREATE TABLE documents ( id TEXT PRIMARY KEY, user_id INTEGER REFERENCES users(id), document_type TEXT, file_path TEXT, extracted_name TEXT, extracted_id_number TEXT, is_verified INTEGER DEFAULT 0, validation_status TEXT, created_at TEXT DEFAULT CURRENT_TIMESTAMP );
```

Table: applications

Stores application submissions submitted to the verifiers queue.

```
CREATE TABLE applications ( id TEXT PRIMARY KEY, user_id INTEGER REFERENCES users(id), service_id INTEGER REFERENCES services(id), form_data TEXT, readiness_score REAL, status TEXT DEFAULT 'pending', officer_notes TEXT, created_at TEXT DEFAULT CURRENT_TIMESTAMP );
```

9. System Security Architecture & Controls

To protect citizen records and prevent fraudulent submissions, the application implements security measures across the frontend, API, and database layers.

Key Security Implementations:

• **CORS Configuration:** The API server configures `cors` options, restricting incoming requests to whitelisted origins. This prevents unauthorized web applications from accessing citizen APIs.

• **Payload Size Limitations:** Express body parser limits are restricted to `1mb` for JSON payloads. This blocks massive payload injection attempts and protects backend endpoints from memory overflow exploits.

• **Role and Privilege Isolation:** The backend enforces strict role-based access control (RBAC). Upon self-registration, user roles default strictly to `'citizen'`. Promotion to `'admin'` (officer) requires manual database entry. All administrative endpoints run validation middleware to block unauthorized actions.

• **Rate Limiting & Authentication Lockouts:**

  • OTP verification requests are rate-limited to prevent brute-force attacks.

  • Temporary tokens are invalidated and blocked after 5 failed verification attempts.

• **SQL Injection Prevention:** Database queries are parameterized (using `$1`, `$2` bindings in postgres/sqlite). Profile updates use whitelisted columns to prevent field-injection exploits.

• **XSS Protection & Sanitization:** To prevent Cross-Site Scripting (XSS), the frontend SPA does not trust HTML strings. It sanitizes dynamically rendered data (like name strings or application IDs) using a utility function (`escapeHTML` in `app.js`) to encode special characters:

```
str.replace(/&/g, '&').replace(//g, '>').replace(/"/g, '"')
```

10. Deployment, Seeding & Operations Guide

This section provides instructions for setting up, seeding, and maintaining the OneCitizen AI application.

Local Setup & Database Seeding:

1. Install Node.js dependencies in the root project directory:

```
npm install
```

2. Run the database seed script to initialize the SQLite database (`backend/one_citizen.db`) and create mock citizen records (including User 2, Gaurav Sujikumar, with a verified Aadhaar card and mock pending applications):

```
node backend/update_seed_run.js
```

3. Launch the development server. The backend runs on port 3000, and static portal routes are served locally:

```
npm run dev
```

Verification Operations:

To test verification states and dashboard updates locally, you can run the automated Playwright test scripts. They will log in using credentials, interact with dashboard elements, and save output logs:

```
python C:\Users\Gaurav Sujikumar\.gemini■ntigravity\brain\...\scratch\test_detail_with_doc.py
```

Vercel Production Deployment:

The project root includes configuration files (`vercel.json`, `vercel-officer.json`) that route portal queries to serverless handlers in production. Every push to the `main` branch on your GitHub repository triggers an automated build on Vercel. In production, ensure the `DATABASE_URL` environment variable is configured in the Vercel dashboard to point to a production database, allowing serverless functions to run correctly.